



UNIVERSIDAD DE LOS LAGOS

ANÁLISIS Y PRE PROCESAMIENTO





UNIVERSIDAD DE LOS LAGOS

CONTENIDO

01 INTRODUCCION

02 ESCALAMIENTO

03 ENCODING

04 CORRELACIÓN

05 RESUMEN



01

INTRODUCCIÓN

ESCALADO DE CARACTERÍSTICAS

INTRODUCCIÓN

Una de las transformaciones más importantes que hay que aplicar a los datos es el **escalado de características**. Salvo algunas excepciones, **los algoritmos de Machine Learning no tienen un buen rendimiento cuando los atributos numéricos de entrada tienen escalas muy diferentes**.



02

ESCALAMIENTO

ESCALADO DE CARACTERÍSTICAS

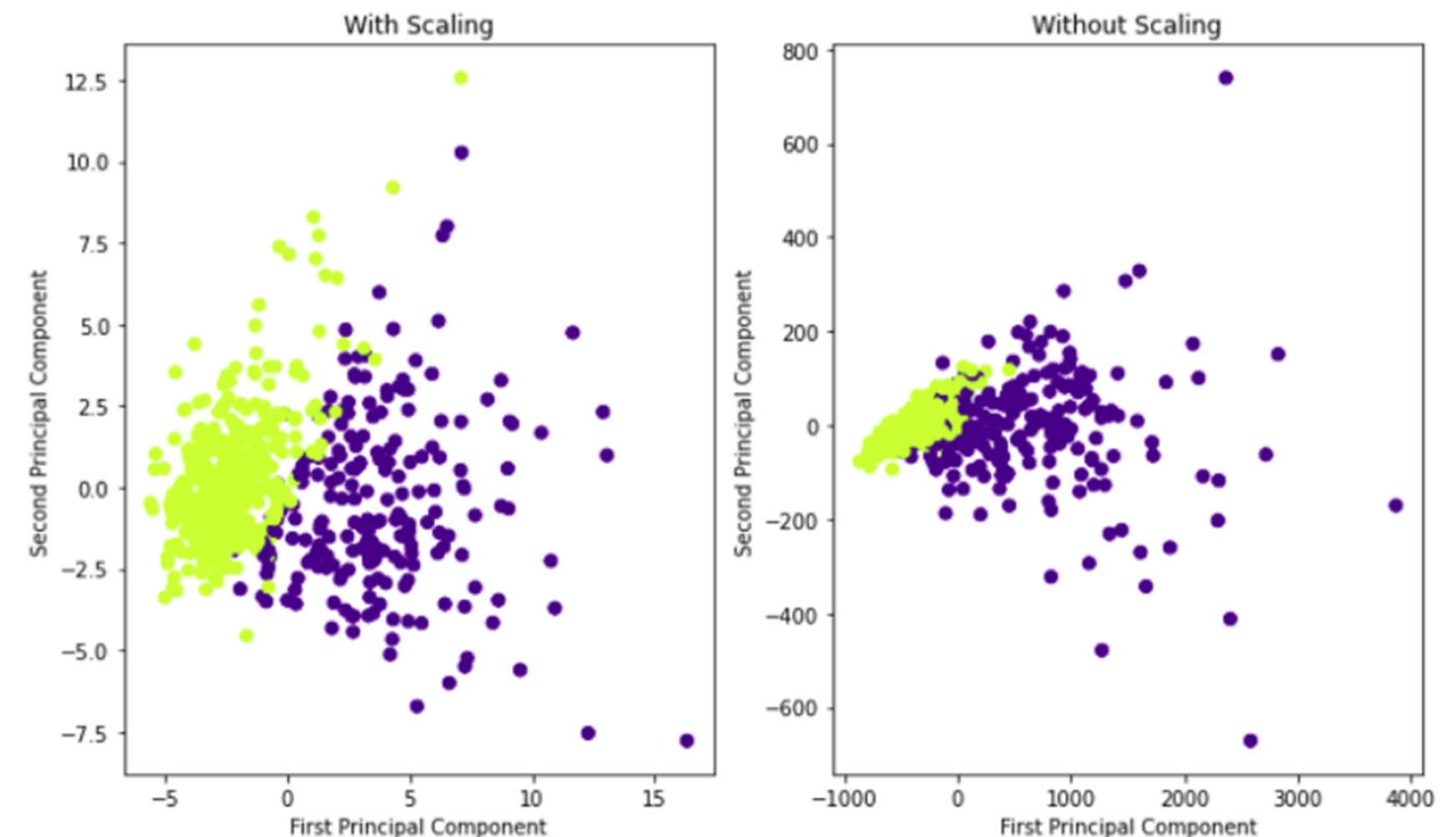
¿POR QUÉ ESCALAR?

La mayoría de las veces, nuestro conjunto de datos contendrá características que **varían mucho en magnitudes, unidades y rango**. Pero dado que la **mayoría de los algoritmos de aprendizaje automático usan la distancia euclidiana entre dos puntos de datos en sus cálculos**, esto es un problema.

Si se dejan solos, estos algoritmos sólo toman en cuenta la magnitud de las características y descuidan las unidades.

Los resultados variarían mucho entre diferentes unidades, por ejemplo entre 5 kg y 5000 g.

Las características con magnitudes altas pesarán mucho más en los cálculos de distancia que las características con magnitudes bajas. Para suprimir este efecto, necesitamos traer todas las características al mismo nivel de magnitudes. Esto se puede lograr escalando.



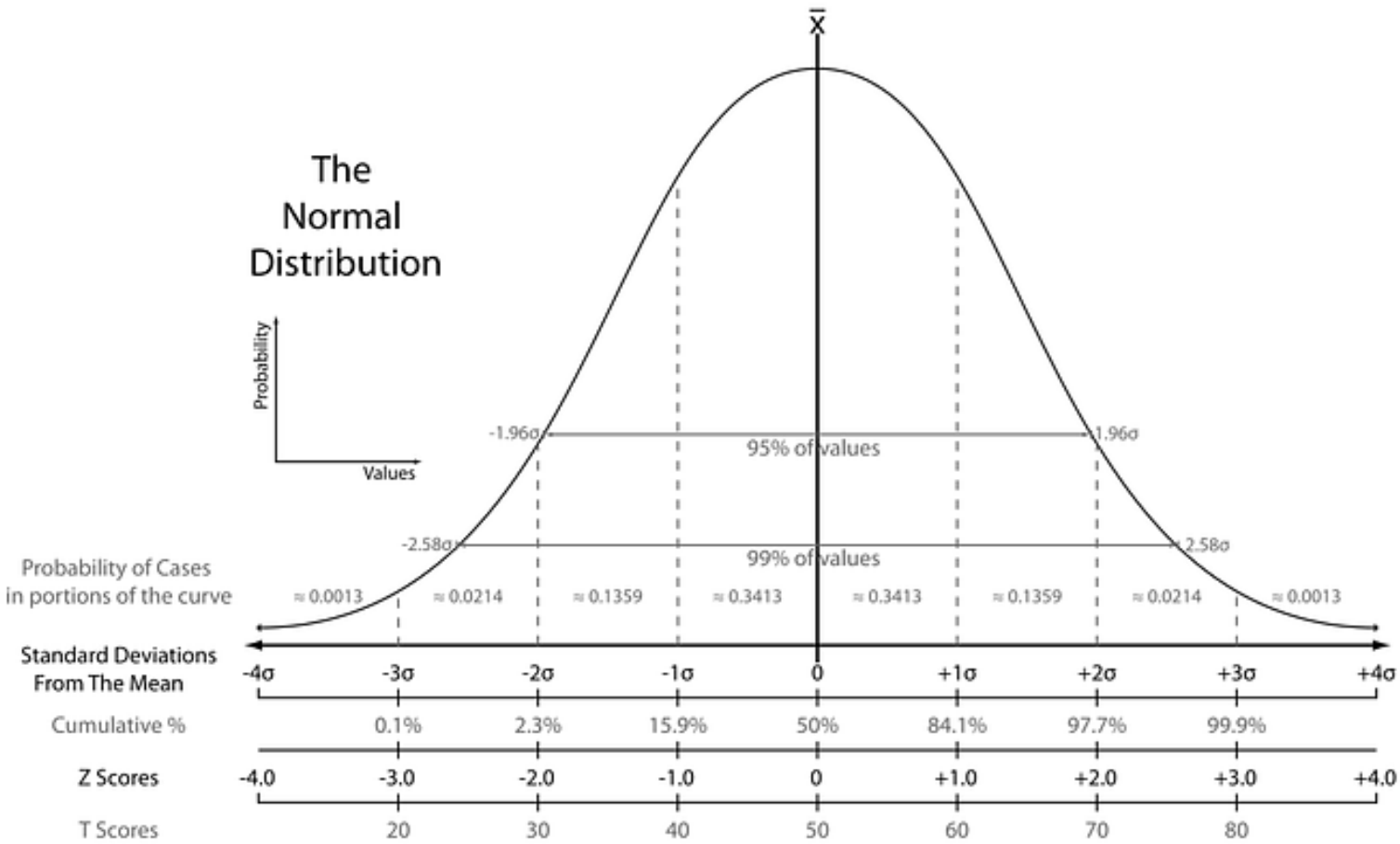
En los gráficos de dispersión mostrados, puede verificar la diferencia entre escalar y no escalar. El rango (escala) es muy amplio sin escala, por lo que es difícil separar puntos. **Cuando se usa escala, proporciona eficiencia y buen rendimiento.**

ESCALADO DE CARACTERÍSTICAS

NORMALIZACIÓN

El objetivo de la normalización es cambiar sus observaciones para que puedan describirse como una distribución normal.

La distribución normal (distribución gaussiana), también conocida como **curva de campana** , es una distribución estadística específica en la que aproximadamente las mismas observaciones caen por encima y por debajo de la media, la media y la mediana son iguales y hay más observaciones más cercanas a la media.



ESCALADO DE CARACTERÍSTICAS

NORMALIZACIÓN MÍNIMO MÁXIMO

Se utiliza la siguiente fórmula.

Podemos ver que cuando $X_i = \min$, entonces $X_{\text{new}} = 0$, y cuando $X_i = \max$, entonces $X_{\text{new}} = 1$.

Esto significa que el valor mínimo de X se asigna a 0 y el valor máximo de X se asigna a 1.

Por lo tanto, **todo el rango de valores de X , desde el mínimo hasta el máximo, se asigna al rango de 0 a 1.**

Esta forma de escalamiento es muy sensible a los outliers.

En Python, usamos MinMaxScaler.

$$X_{\text{new}} = \frac{X_i - \min(X)}{\underset{\text{Referencia}}{\max(x) - \min(X)}}$$

ESCALADO DE CARACTERÍSTICAS

ESTANDARIZACIÓN

La **estandarización** (también llamada **normalización de puntuación z**) transforma sus datos de modo que la distribución resultante tenga una media de 0 y una desviación estándar de 1.

En Python usamos la función `StandardScaler`.

Es más resistente a **outliers**, pero no es muy interpretable para datos que se alejan mucho de una distribución Normal.

$$X_{\text{new}} = \frac{X_i - X_{\text{mean}}}{\text{Standard Deviation}}$$

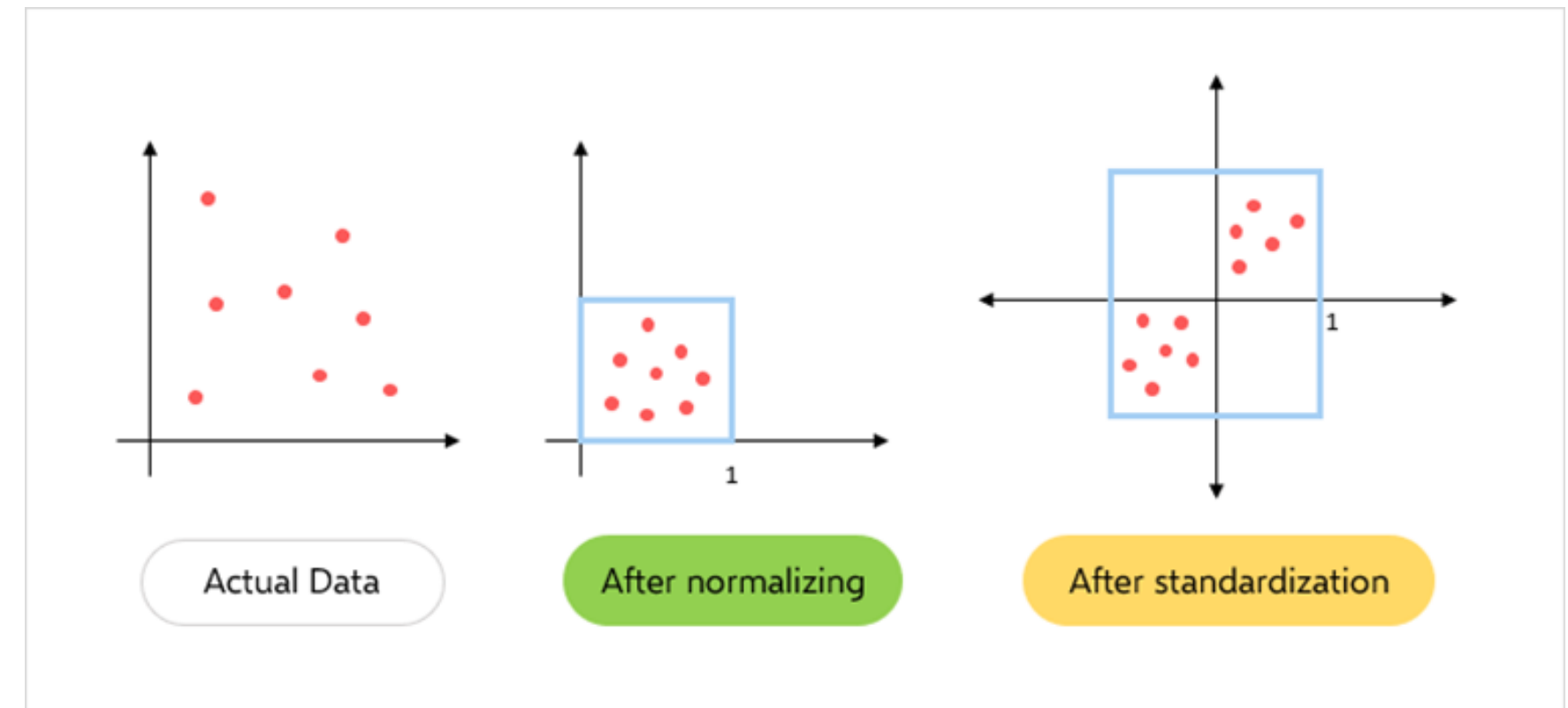
Referencia

ESCALADO DE CARACTERÍSTICAS

COMPARACIÓN

Escalado vs. Normalización: ¿Cuál es la diferencia? En ambos casos, está transformando los valores de las variables numéricas para que los puntos de datos transformados tengan propiedades útiles específicas.

La diferencia es que, al escalar, está cambiando el rango de sus datos mientras que en la normalización está cambiando la forma de la distribución de sus datos.





03

ENCODING



INTRODUCCIÓN

Sabemos que existen **variables categóricas ordinales y nominales**.

La mayoría de los algoritmos de aprendizaje automático no pueden manejar variables categóricas a menos que las convirtamos en valores numéricos.

El **rendimiento de muchos algoritmos varía en función de cómo se codifican las variables categóricas**.

Hay muchas maneras en que podemos codificar estas variables categóricas como números y usarlas en un algoritmo.

ENCODING



ONE HOT ENCODING

En este método, asignamos cada categoría a un vector que contiene 1 y 0, lo que denota la presencia o ausencia de la característica.

El número de vectores depende del número de categorías de características.

Este **método produce muchas columnas que ralentizan significativamente el aprendizaje** si el número de la categoría es muy alto para la función.

```
from sklearn.preprocessing import OneHotEncoder
ohc = OneHotEncoder()
ohe = ohc.fit_transform(df.Temperature.values.reshape(-1,1)).toarray()
dfOneHot = pd.DataFrame(ohe, columns = ["Temp_"+str(ohc.categories_[0][i])
                                     for i in range(len(ohc.categories_[0]))])

dfh = pd.concat([df, dfOneHot], axis=1)
dfh
```

	Temperature	Color	Target	Temp_Cold	Temp_Hot	Temp_Very Hot	Temp_Warm
0	Hot	Red	1	0.0	1.0	0.0	0.0
1	Cold	Yellow	1	1.0	0.0	0.0	0.0
2	Very Hot	Blue	1	0.0	0.0	1.0	0.0
3	Warm	Blue	0	0.0	0.0	0.0	1.0
4	Hot	Red	1	0.0	1.0	0.0	0.0
5	Warm	Yellow	0	0.0	0.0	0.0	1.0
6	Warm	Red	1	0.0	0.0	0.0	1.0
7	Hot	Yellow	0	0.0	1.0	0.0	0.0
8	Hot	Yellow	1	0.0	1.0	0.0	0.0
9	Cold	Yellow	1	1.0	0.0	0.0	0.0

ENCODING

BINARY ENCODING

La codificación binaria convierte una categoría en dígitos binarios.

Cada dígito binario crea una columna de características.

Si hay n categorías únicas, la codificación binaria da como resultado las únicas funciones de registro $(base\ 2)^n$.

En este ejemplo, tenemos cuatro características; por lo tanto, las características codificadas en binario serán tres características.

En comparación con One Hot Encoding, **esto requerirá menos columnas de funciones** (para 100 categorías, One Hot Encoding tendrá 100 funciones, mientras que para la codificación binaria, necesitaremos solo siete funciones).

```
import category_encoders as ce
encoder = ce.BinaryEncoder(cols=['Temperature'])
dfbin = encoder.fit_transform(df['Temperature'])
df = pd.concat([df, dfbin], axis=1)
df
```

	Temperature	Color	Target	Temperature_0	Temperature_1	Temperature_2
0	Hot	Red	1	0	0	1
1	Cold	Yellow	1	0	1	0
2	Very Hot	Blue	1	0	1	1
3	Warm	Blue	0	1	0	0
4	Hot	Red	1	0	0	1
5	Warm	Yellow	0	1	0	0
6	Warm	Red	1	1	0	0
7	Hot	Yellow	0	0	0	1
8	Hot	Yellow	1	0	0	1
9	Cold	Yellow	1	0	1	0

1. Las categorías se convierten primero en orden numérico a partir de 1 (el orden se crea a medida que las categorías aparecen en un conjunto de datos y no significan ninguna naturaleza ordinal).
2. Luego, esos números enteros se convierten en código binario, por ejemplo, 3 se convierte en 011, 4 se convierte en 100.
3. Luego, los dígitos del número binario forman columnas separadas.

LABEL ENCODING

En esta codificación, a cada categoría se le asigna un valor de 1 a N (donde N es el número de categorías para la función).

Un problema importante con este enfoque es que no hay relación ni orden entre estas clases, pero el algoritmo podría considerarlas como algún orden o alguna relación.

En el siguiente ejemplo, puede verse como Scikit Learn nos permite aplicar Label Encoding con:

(Cold<Hot<Very Hot<Warm....0 < 1 < 2 < 3) .

```
from sklearn.preprocessing import LabelEncoder
df['Temp_label_encoded'] = LabelEncoder().fit_transform(df.Temperature)
df
```

	Temperature	Color	Target	Temp_label_encoded
0	Hot	Red	1	1
1	Cold	Yellow	1	0
2	Very Hot	Blue	1	2
3	Warm	Blue	0	3
4	Hot	Red	1	1
5	Warm	Yellow	0	3
6	Warm	Red	1	3
7	Hot	Yellow	0	1
8	Hot	Yellow	1	1
9	Cold	Yellow	1	0



ORDINAL ENCODING

Realizamos la codificación ordinal para **garantizar que la codificación de las variables conserve la naturaleza ordinal de la variable**. Esta codificación se ve casi similar a Label Encoder, pero es ligeramente diferente, ya que la Label Encoder no consideraría si la variable es ordinal o no, y asignará una secuencia de números enteros.

En este código usando Pandas, primero debemos asignar el orden original de la variable a través de un diccionario. Luego podemos mapear cada fila para la variable según el diccionario.

Aunque es muy sencillo, **requiere codificación** para indicar los valores ordinales y la asignación real de texto a un número entero según el orden.

```
Temp_dict = { 'Cold' : 1,
              'Warm' : 2,
              'Hot' : 3,
              'Very Hot' :4}
df['Temp_Ordinal'] = df.Temperature.map(Temp_dict)
```

	Temperature	Color	Target	Temp_Ordinal
0	Hot	Red	1	3
1	Cold	Yellow	1	1
2	Very Hot	Blue	1	4
3	Warm	Blue	0	2
4	Hot	Red	1	3
5	Warm	Yellow	0	2
6	Warm	Red	1	2
7	Hot	Yellow	0	3
8	Hot	Yellow	1	3
9	Cold	Yellow	1	1



04

CORRELACIÓN

CORRELACIÓN

COEFICIENTE DE CORRELACIÓN

El coeficiente de correlación es una medida estadística mediante la cual podemos determinar objetivamente la relación entre 2 variables de datos.

En estadística, existen varios métodos para determinar el coeficiente de correlación.

Existen principalmente 2 métodos para determinar el coeficiente de correlación:

Coeficiente de correlación de Pearson

Coeficiente de correlación de rangos de Spearman

Nos centraremos, por ahora, en el primero de ellos.

$$r = \frac{\sum (x_i - \bar{x}) (y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

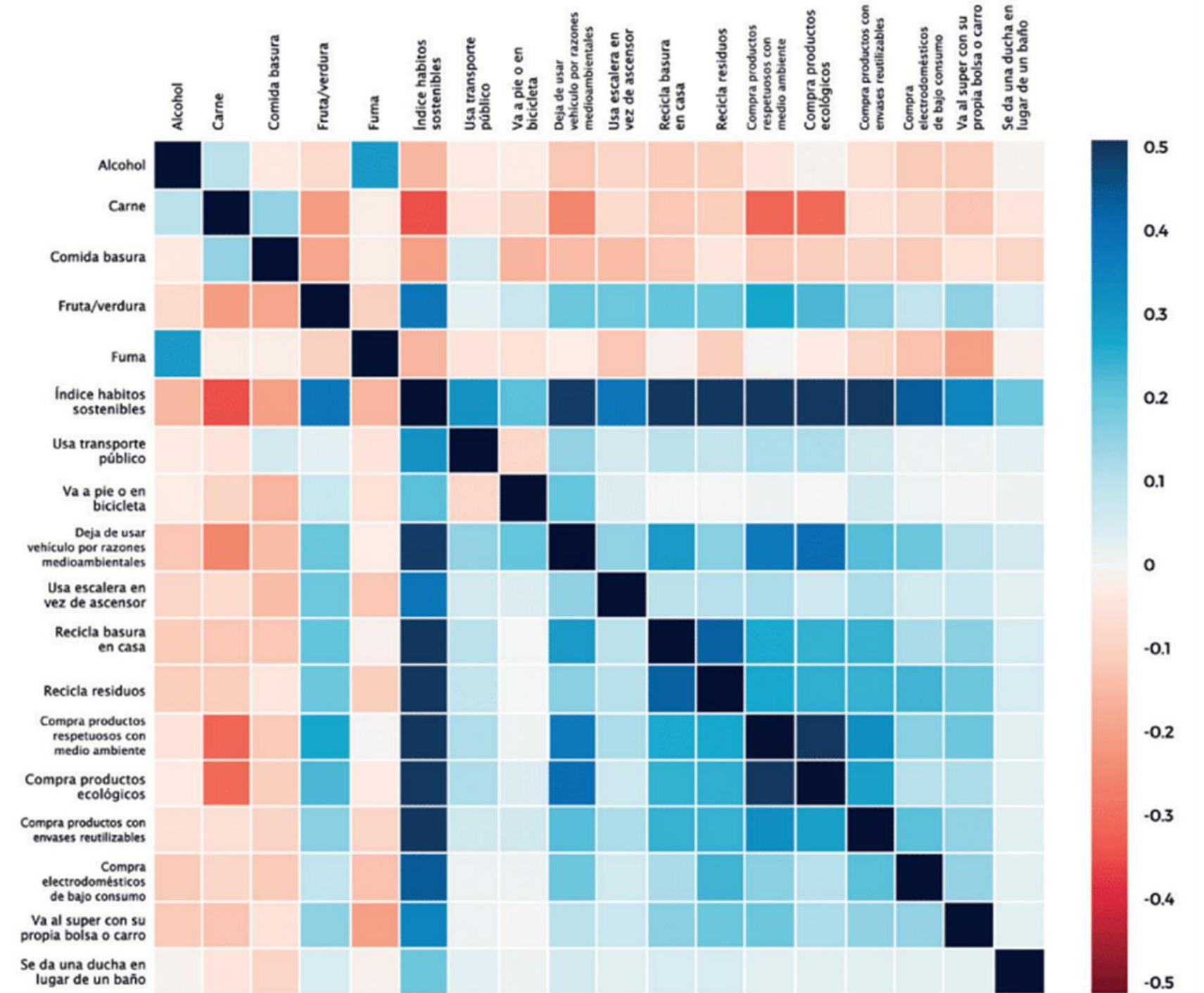
Coeficiente de Correlación de Pearson

COEFICIENTE DE CORRELACIÓN

El coeficiente de correlación de Pearson siempre se encuentra entre $+1$ (correlación positiva perfecta) y -1 (correlación negativa perfecta); 0 indica que no hay correlación.

Las variables pueden tener una asociación que no sea lineal, en cuyo caso el coeficiente de correlación puede no ser una métrica útil.

A partir de una tabla de correlaciones, se puede trazar un mapa de calor para mostrar visualmente la relación entre múltiples variables.



CORRELACIÓN

COEFICIENTE DE CORRELACIÓN

Ahora, antes de calcular el coeficiente de correlación de Pearson, debemos comprender que esta fórmula supone algunos supuestos.

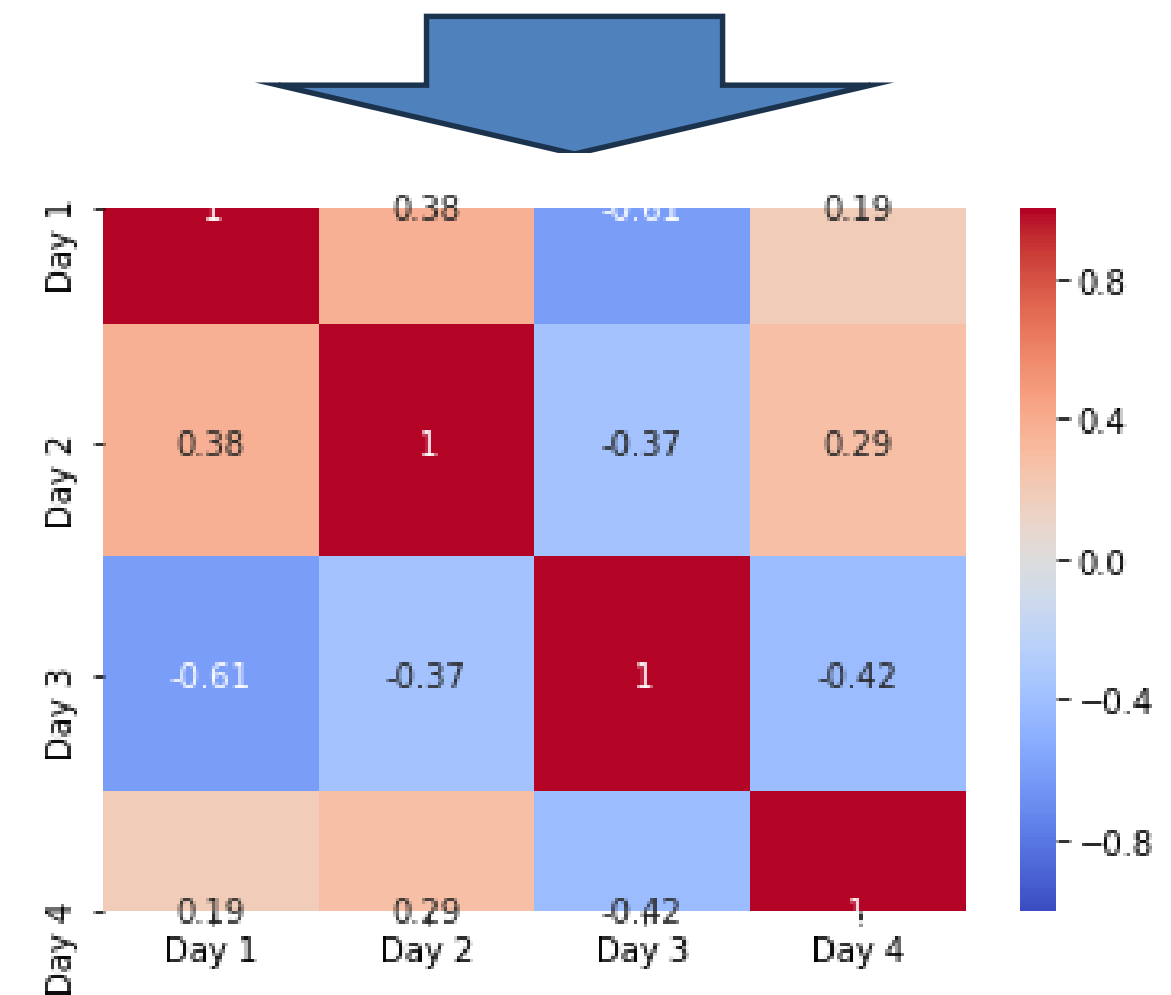
Estos supuestos deben ser ciertas si queremos calcular el coeficiente de correlación utilizando esta fórmula.

- Relación lineal: debe haber una relación lineal entre 2 variables.
- Distribución normal: ambas variables deben seguir aproximadamente la distribución normal.
- Pares relacionados: cada observación en el conjunto de datos debe ser un par de valores.
- Sin valores atípicos: no debería haber valores atípicos extremos en el conjunto de datos.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.DataFrame({
    "Day 1": [7, 1, 5, 6, 3, 10, 5, 8],
    "Day 2": [1, 2, 8, 4, 3, 9, 5, 2],
    "Day 3": [4, 6, 5, 8, 6, 1, 2, 3],
    "Day 4": [5, 8, 9, 5, 1, 7, 8, 9],
})

sns.heatmap(df.corr(), vmin=-1, vmax=+1, annot=True, cmap="coolwarm")
```



Mapa de calor generado con la Librería SEABORN



05

RESUMEN



RESUMEN

- ❑ Hemos visto técnicas de ESCALADO para características numéricas.
- ❑ También, Features Encoding para transformar variables categóricas.
- ❑ Finalmente, análisis de correlación utilizando el Coeficiente de Correlación de Pearson.